

Neural Search System

Team:

- Giorgi Beridze
- Sopho Gurjishvili
- Mariam Kakhidze
- Elene Kvitsiani

GitHub link: <https://github.com/ekvit22/neural-search-system-nlp>

We built a neural semantic search system. The system takes a question that the user asks in natural language (a query) and returns the most relevant text passages from the corpus, following this principle:

query → Transformer encoder → query embedding → cosine similarity → top-k chunks

The core of the system is a text encoder based on the Transformer architecture, which we implemented and trained ourselves without using 'sentence-transformers' or any pretrained weights. The model maps any question or text into a shared 256-dimensional vector space. Training was done with contrastive learning, so that questions and answers with similar meaning end up close to each other in that space. Search is carried out with cosine similarity, following the nearest neighbour principle.

We trained the model on MS MARCO data and compared it against two classic lexical baselines (BM25 and TF-IDF). For evaluation we used the Recall@k and MRR metrics on a held-out test set. We also prepared a working demo for searching over 200 to 300 word passages from the book "Speech and Language Processing" by Dan Jurafsky and James Martin.

Our goal was not to reach state-of-the-art results, but to understand the task in depth, justify our technical choices and analyse the results we obtained objectively, including explaining why our neural model fell behind the baselines.

Data

To train the model we used MS MARCO v2.1. This dataset consists of real Bing search queries, each of which comes with candidate passages in which human annotators have marked the direct answer (`is_selected=1`). This gives exactly the supervision signal that a retrieval model needs.

We downloaded the first 50,000 training examples. After deduplication we obtained 1,898,742

unique passages (the full corpus) and 122,356 queries with at least one positive answer. We did not train the model directly on Jurafsky's book. MS MARCO is far better suited for learning general question-answer matching, while the book is used only for the final evaluation. Training on the book would also have led to overfitting.

In the training data, a query is a short question typed by a user, for example "what is the transformer attention mechanism", and a passage is a paragraph-length text fragment that contains a potential answer. The model converts both into vectors with the same encoder, and relevance is measured with cosine similarity.

Train / Validation / Test

The data was split by query in an 80/10/10 ratio with a fixed random seed: Train, 97,884 triplets for training the encoder; Validation, 12,235 queries for monitoring the loss and selecting the best checkpoint; Test, 12,237 queries for the final evaluation.

Splitting by query guarantees that no test query is seen by the model during training, so we measure the ability to generalise rather than memorise.

For each training query we built a (query, positive, hard-negative) triplet: the positive is the passage with `is_selected = 1`, and the hard negative is a non-selected passage from the same MS MARCO example. Because the negative passage is taken from the same search results, it is topically close to the query but does not answer it. This forces the model to learn subtle semantic differences instead of simple keyword overlap. From random negative passages the model would learn almost nothing.

Baseline models

We decided to compare our model against two classic lexical retrieval models ('eval/baselines.py'):

- BM25 (Okapi BM25, using the 'rank_bm25' library): the industry standard in information retrieval. It counts word overlap, gives more weight to rare terms and applies term frequency saturation. It also normalises for document length, which prevents long passages from getting an unfair advantage in the search system
- TF-IDF (unigrams and bigrams, scikit-learn, sublinear TF, no stop words): cosine similarity between the query and document vectors.

Both baselines index the entire corpus.

Model architecture

The Transformer encoder was written entirely from scratch ('model/model.py'). For token embeddings we used an 8,000-token BPE tokenizer that we trained ourselves on MS MARCO ('data/build_tokenizer.py'), together with fixed sinusoidal positional encoding. This is necessary because a Transformer otherwise has no way of knowing which word comes after which, and this mechanism is how we teach the model to perceive word order. BPE fits well here because it finds a good middle ground between vocabulary size and coverage: frequent words remain compact, single tokens (short sequences, fast encoding), while rare or unknown words are broken into subwords instead of turning into <UNK>. In this way any input is fully covered without an oversized vocabulary, including domain-specific MS MARCO terms and morphologically rich languages (Georgian, for example). The size of 8,000 keeps the embedding table small, which suits a compact model trained from scratch. We also used a 4-layer pre-norm Transformer block with Multi-Head Self-Attention ($d_{\text{model}} = 256$, 4 heads), GELU activation ('dim_ff = 512'), residual connections and dropout (0.1). Self-Attention lets each token pull context from the whole sequence, including long-range dependencies. This is exactly what makes semantic matching possible: the model can place "automobile" and "car" close to each other, whereas keyword-based methods treat them as completely different terms. The Multi-Head mechanism (4 heads) lets the model pick up different kinds of relationships in parallel. We implemented Masked Mean Pooling, which leaves padding out of the calculation, to obtain a fixed 256-dimensional vector. This approach takes one average meaning vector from the words of a passage while skipping the empty positions. Masking the padding is essential so that empty tokens do not distort the average.

The architecture is of the bi-encoder type: the same weights encode both the query and the document, which makes their vectors directly comparable. A cross-encoder approach is unsuitable for a search system, because the model could not re-score millions of passages online.

Contrastive learning

As the loss function we used Triplet InfoNCE Loss ('model/contrastive_learning.py'). With this function, at every step of training the model compares one specific query not against one passage but against dozens of different passages at once. The model does its best to pull the vector of the correct answer towards itself and increase the cosine similarity, while pushing every incorrect answer as far away in the space as possible. For a batch of size B , each query is scored against $2B$ candidates (its own positive + its own negative + the positives and negatives of the other queries in the batch), where the correct positive is treated as the target class in cross-entropy (temperature = 0.05). The in-

batch negatives mechanism is what makes this approach especially effective: in a batch of size B each query is scored against $2B$ candidates essentially for free, since these vectors have already been computed. More negatives give a stronger gradient signal, and that in turn gives a sharper partition of the space. The low temperature (0.05) sharpens the softmax, which forces the model to push away even the most similar incorrect candidates. In addition, the specially chosen hard negative teaches the nuances, while the in-batch negatives teach the broad distinctions. InfoNCE also pushes the vectors of incorrect answers away in all directions, which keeps the data from collapsing together in the space by mistake.

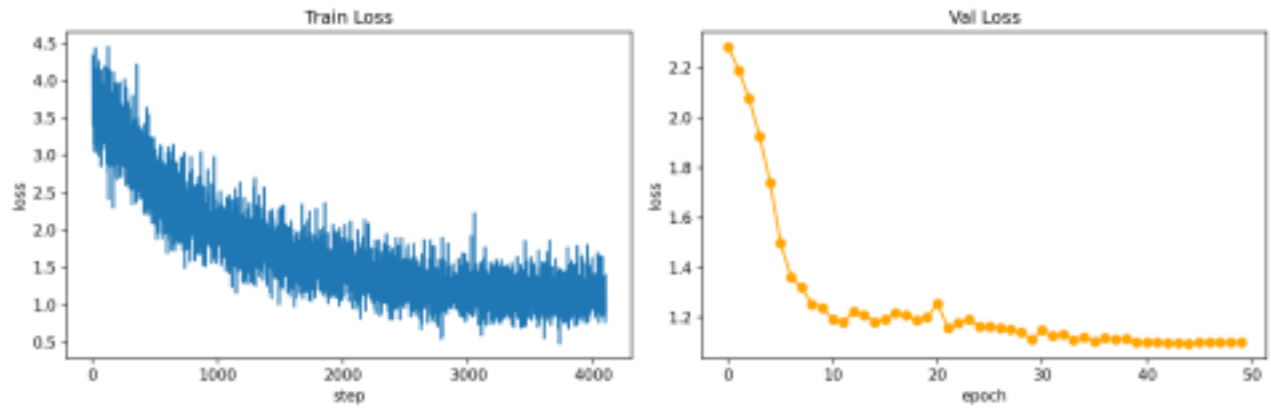
Experiments

We ran two experiments.

In the first attempt we decided to set standard parameters for the model, such as batch size 64 and LR 0.0001. Otherwise, if we had picked numbers at random and the training had not gone the way we wanted, it would have been very hard to figure out where the mistake was: in the code, in the data, in the architecture or somewhere else. We launched this model to make sure that the learning process would start correctly, which would indicate that the fundamental mechanisms (tokenisation, Transformer blocks and the contrastive loss) were set up properly. Contrastive learning is a fairly sensitive process and we needed to see how the model reacted in a simple and controlled setting. We used 50,000 examples (a small amount) so that the drop in loss would show up in a short time. The result was that training went normally and nothing errored out, but at the eighth epoch the model reached a convergence state and stopped learning, which told us that it had exhausted the data and for that reason could no longer learn.

To fix this we tried to increase the amount of data as much as we could, raising it from 50k to 200k, and we increased the batch size to 128, giving the model a better chance to tell the real signal apart from plain noise. As a result, in the second run the model managed to reach a considerably lower loss, which indicates that the embeddings generated by the model became more accurate and more informative. The model was again learning cleanly and stably, and the decrease in `val_loss` was monotonic, unlike in the previous run. The model was trained for 15 epochs, and `train_loss` and `val_loss` remained balanced after the epochs were completed. It should also be noted, however, that from epoch 12 onward the improvement in `val_loss` practically stopped, which indicates that the model had exhausted its architectural potential. At this point adding more data would not have given us any benefit, since the capacity of the model itself is what limits it at this stage.

Run 1 gave us the following result:

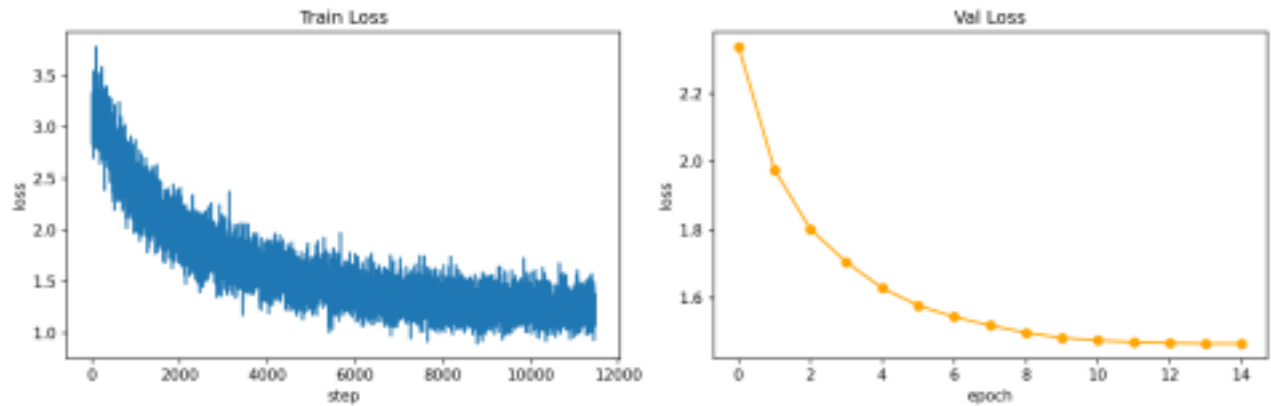


The parameters of Run 2 were:

- Optimizer: AdamW
- Learning Rate: $2e^{-4}$
- LR schedule: Linear warmup (10%) -> Cosine decay
- Batch Size: 128
- Epochs: 15
- Temperature: 0.05
- Max. length: 128 tokens
- Mixed Precision: AMP (fp16) on GPU

Logging was done in TensorBoard. The validation loss decreased steadily across epochs with no signs of overfitting:

Epoch	loss
1	2.338
3	1.801
5	1.628
7	1.544
9	1.497
11	1.475
13	1.467
15	1.465



On the graph we can see that the training loss falls steadily from 3.5 to 1.0, while the validation curve flattens out around 1.46, which confirms that the model converged.

Evaluation

For evaluation we used the following metrics:

- Recall@k (k = 1, 5, 10): the share of queries for which the correct answer landed in the top k results.
- MRR (Mean Reciprocal Rank): measures how high (at what position) the model returned the correct answer. The closer to 1, the better.

Fully vectorising the entire corpus was impossible with our limited resources. So we built a reduced evaluation corpus of 112,229 passages: it contains every correct passage for the test queries plus 100,000 random distractors. Testing was carried out on 500 randomly chosen test queries for all three models.

Results

	Recall@1	Recall@5	Recall@10	MRR
BM25	0.4540	0.6900	0.7680	0.5575
TF-IDF	0.4060	0.6620	0.7420	0.5154
Ours	0.2640	0.4420	0.5040	0.3384

The lexical baselines beat our neural model. BM25 turned out to be the strongest (Recall@10: 0.768 vs our 0.504). The reason is that in MS MARCO the queries and answers very often share the exact same keywords. Lexical matching is an extremely strong signal here. For dense models to beat BM25 they need long pretraining or distillation from a cross-encoder, which we did not have. On top of that, our model is small (4 layers, 256 dimensions, 50,000 examples).

The model did learn to build a semantic space: the loss decreased and eventually stabilised and we reached convergence, meaning that our model's architecture and code are correct and both the training and the demo worked properly, but its capacity turned out to be insufficient to beat the classic algorithms.

For the demo we downloaded 3 chapters of the book "Speech and Language Processing" (text processing, vector semantics and Transformers). The text was split into 132 chunks, each roughly 200-300 words.

Our encoder indexed these chunks. On semantic questions there is a good match: for example, for the query "how are word embeddings learned" the model successfully returns the description of the static embedding matrix. It is somewhat weaker on complex questions, which is consistent with the results we obtained. The demo is fully reproducible in the 'evaluation.ipynb' notebook.